



CPSC 100

Computational Thinking

Intro to Programming

Instructor: Firas Moosvi
Department of Computer Science
University of British Columbia



Agenda

- Course Admin
- Intro to Snap!
- Components of an Algorithm

Course Admin

Course Admin

- **Apologies for the delays on Test and Lab grading - we'll get these out to you as soon as possible!**

Course Schedule

This is the tentative plan for CPSC 100 this term. Things may change depending on many factors so this is subject to change.

- **Reminder of the Course Schedule**

Week	Week of	Topic	LL	Lab	Project	Test (Content)
1	Jan 5	Introductions	LL0	-		-
2	Jan 12	Algorithms I	LL1	Lab 1		T1 (Intro)
3	Jan 19	Algorithms II	LL2	Lab 2	PM1	T2 (Alg I)
4	Jan 26	Data Representation	LL3	Lab 3		T3 (Alg II)
5	Feb 2	Programming I	LL4	Lab 4	PM2	T4 (Data Rep)
6	Feb 9	Programming II	LL5	Lab 5		T5 (Programming I)
7	Feb 16	Reading week	-	-		-
8	Feb 23	Internet and HCI	LL6	Lab 6	PM3	T6 (Programming II)
9	Mar 2	Artificial Intelligence I	LL7	Lab 7		T7 (Internet & HCI)
10	Mar 9	Artificial Intelligence II	LL8	Lab 8	PM4	T8 (AI I)
11	Mar 16	Catch-up week	-	-		-
12	Mar 23	Data Mining	LL9	Lab 9		T9 (AI II)
13	Mar 30	Data Visualization	LL10	Lab 10	PM5	T10 (Data mining)
14	Apr 6	Final Projects	LL11	-		T11 (Data Visualization)



CPSC 100 Term Project

- The goal of the CPSC 100 term project is for you to create a “digital artifact”: an **infographic/video/learning with lyrics!** to help explain a topic that touches on several parts of the CPSC 100 course.
- Projects will be done in groups of 3 or 4, **groups should be formed now**
 - There will be group ***and*** individual components



CPSC 100 Term Project

- Project component is 25% of your CPSC 100 grade
- There will be 5 deliverables called “Milestones”
 - You will get one final project score at the end
 - Milestones will build on top of each other
 - Feedback will be given for each milestone to help you improve



CPSC 100 Term Project

Milestones	Purpose	Due Date
Milestone 1	Group Formation & Team Contract	February 4, 2026 at 6 PM
Milestone 2	Proposal & Sketch	
Milestone 3	Individual Contributions	
Milestone 4	Group Contribution	
Milestone 5	Final Presentation & Peer Review	

Algorithms

An ***algorithm*** describes a sequence of steps that is:

1. Unambiguous

- No “assumptions” are required to execute the algorithm
- The algorithm uses precise instructions

2. Executable

- The algorithm can be carried out in practice

3. Terminating

- The algorithm will eventually come to an end, or halt



Components of an Algorithm

An ***algorithm*** is a precise, systematic method for producing a specified result.

In 1966 it was proved (*structured program theorem*) that any algorithm can be made with only three “ingredients”:

1. Sequencing
2. Selection
3. Iteration

Bread Making Algorithm



Components of an Algorithm - Bread Making

Repeat 10 times:

1. Preheat oven (400° C)
2. Combine ingredients in bowl to form dough
3. Put dough into bread pan
4. If ingredients contain yeast, allow to sit at room temperature for 1 hour
5. Put bread pans into preheated oven and bake for 30 minutes

Task: In this algorithm, where do you see the three components?

Algorithms to Programming!



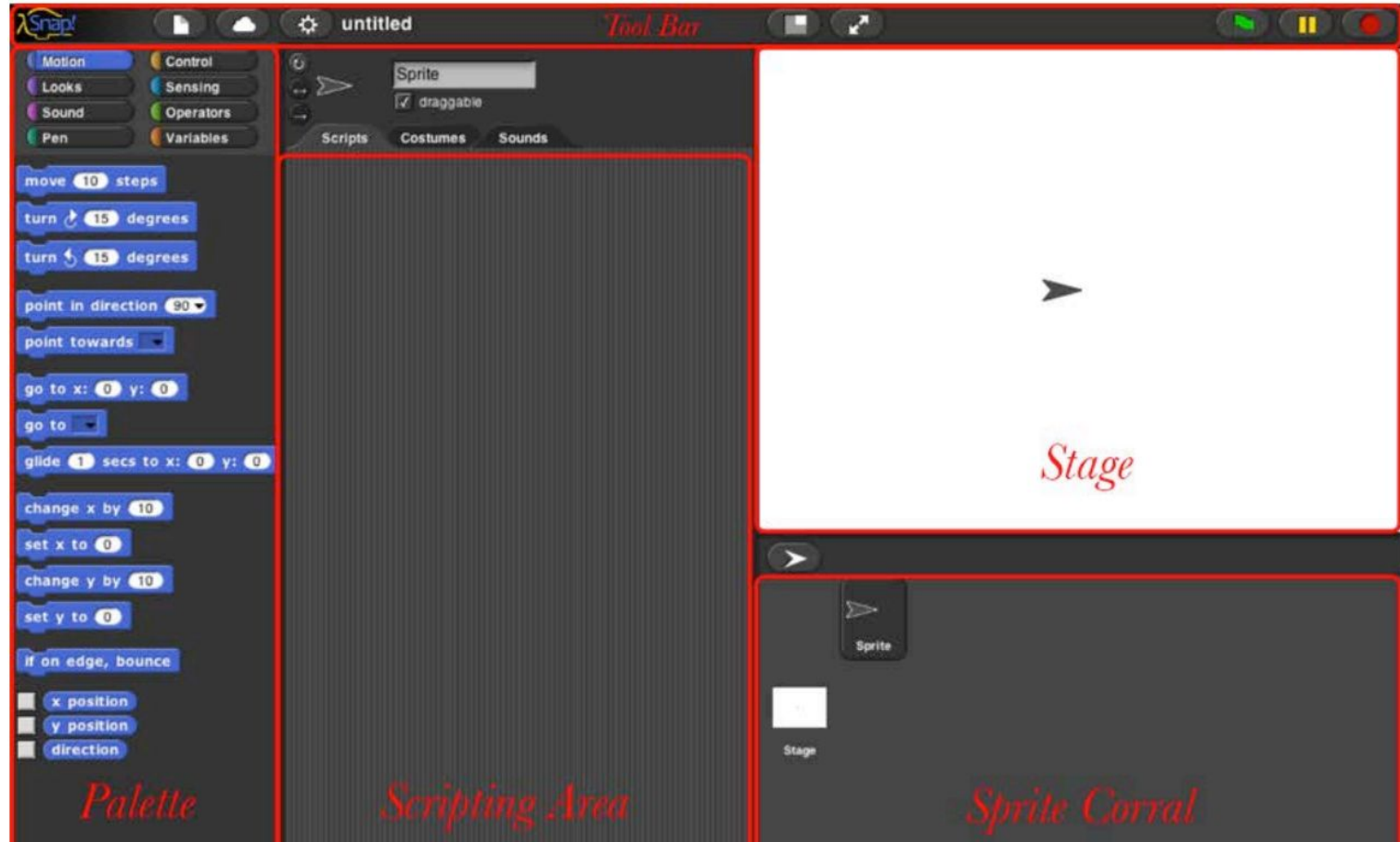
Learning Goals

After this week's lecture, you should be able to:

- Learn about Snap! and its interface
- **Use Snap! blocks** to represent algorithms
- Be able to **trace** through code using sequences of instructions, variables, loops, and conditional statements in short programs
 - Read carefully: it says be able to **trace** code, **not write code**. In order to help you do this, you will write a small amount of code in lab. You will not, however, be asked to write code on exam.
- Describe in English what a block of *Snap!* code does.
- **Identify and describe** the components of an algorithm
 - (i.e., sequencing, selection, and iteration)

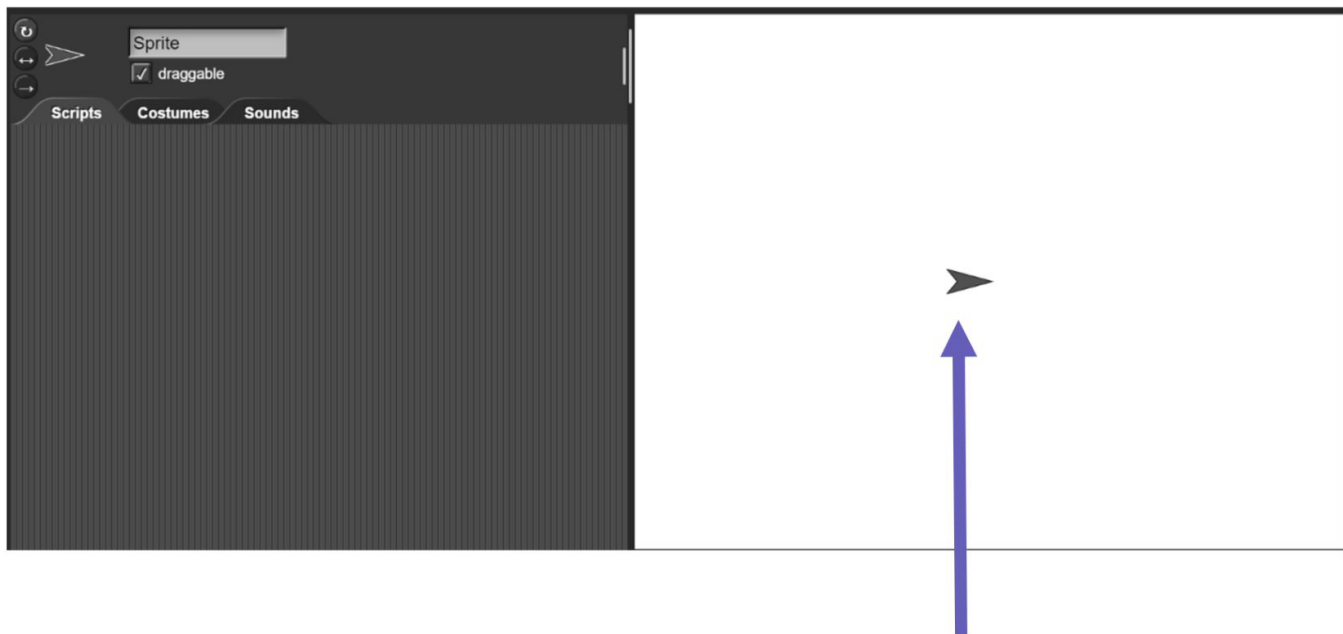
Intro to Snap!

SNAP! Interface



Intro to Snap!

We call our screen our “**stage**”. “Things” we add are called **sprites**. A sprite is an object you can move on a larger scene.

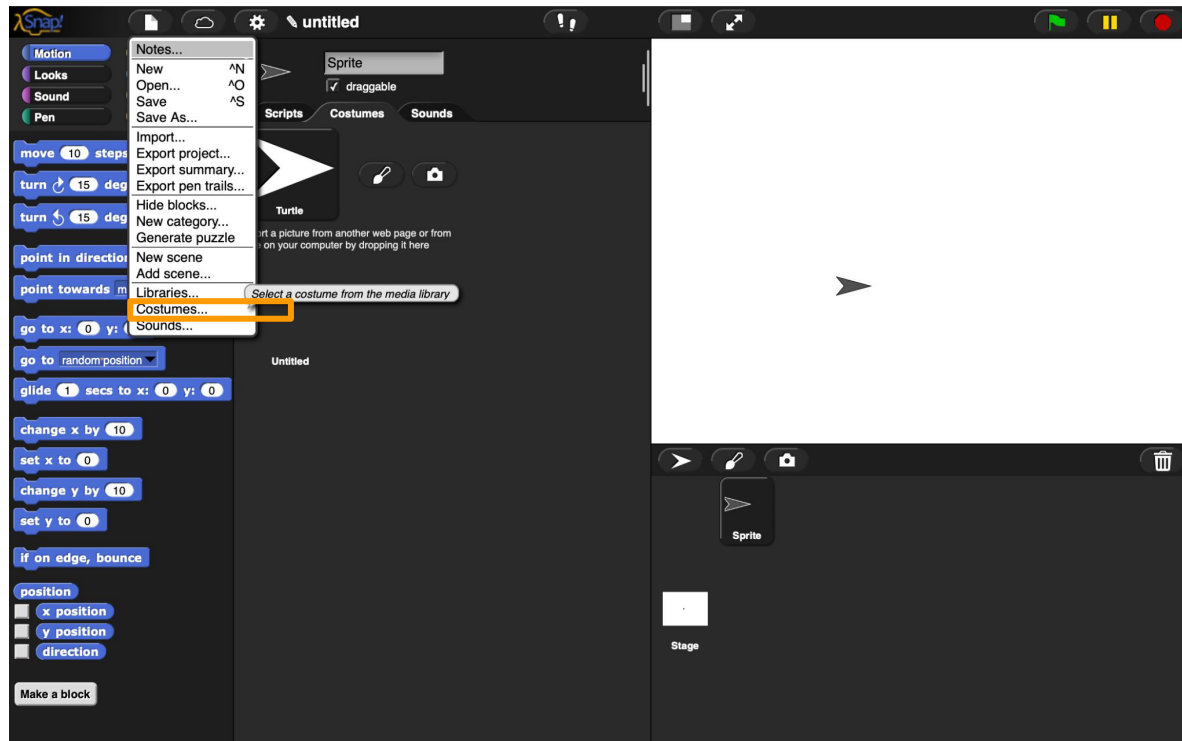


Code area (currently blank)

Sprite

Intro to Snap!

Let's change our Sprite's costume:



Stage

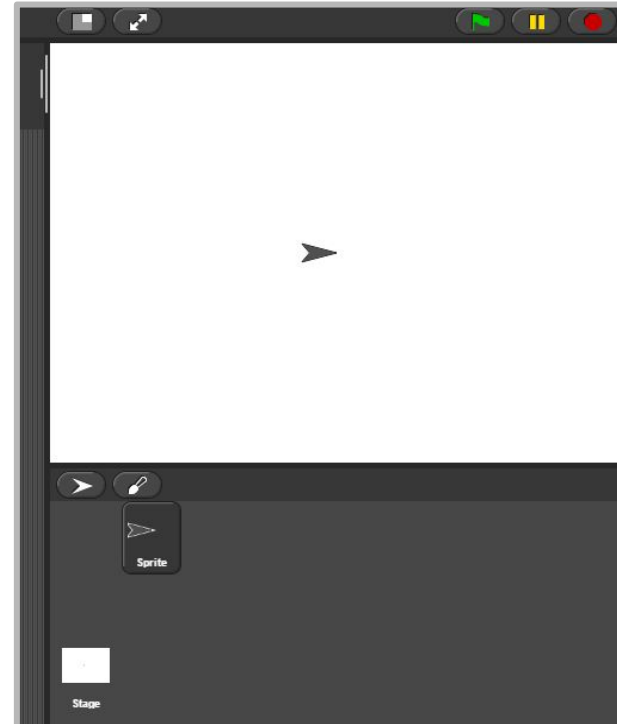
(aka the background)



The stage/background is 480 pixels by 360 pixels.
These can be called either pixels or steps.
The stage size can be changed in settings.

The locations on the stage are just like x- and y-coordinates in math. The center of the stage is (0,0).

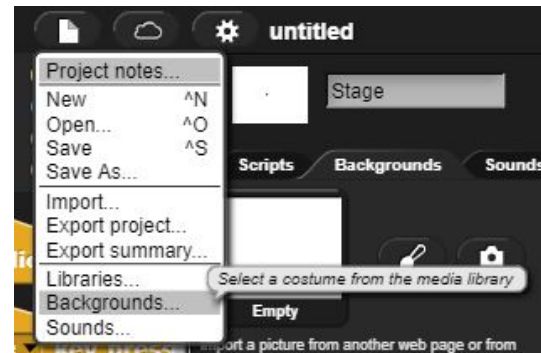
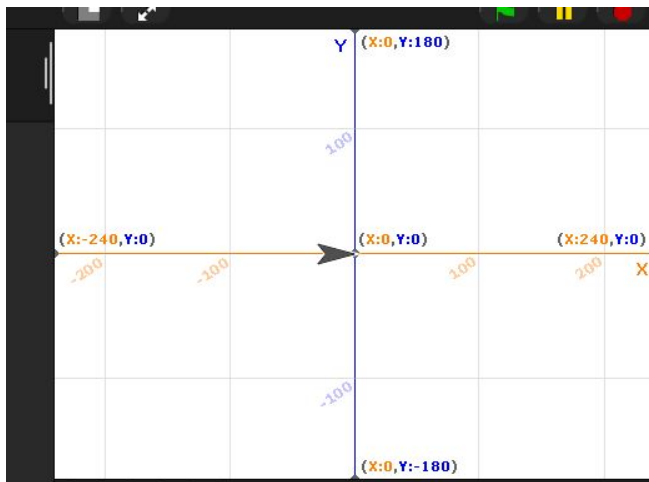
The top right corner is (240,180), the bottom right corner is (240,-180) ... and so on for each corner.



Helpful Background

(aka the XY coordinate plane)

1. Select the "Stage" in the bottom right of the screen.
2. Go to the top menu, and select:
File Menu → Backgrounds ...
3. Select "XY Grid", which is the last one.



Control Blocks

Control blocks are used to manage how your program runs. These control when portions of code -- start, stop, repeat, etc



Some simple examples:

1. "When green flag is clicked, run ____"
2. "Repeat _____ 3 times"
3. "When spacebar is pressed, stop ____"

wait until

when green flag clicked

repeat 3

wait 1 secs

when space key pressed

if
else

broadcast and wait

if key space pressed?
stop all

forever

Pen Blocks

Use the Pen to draw.

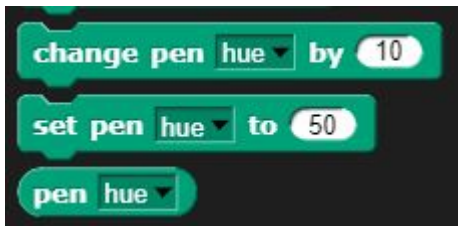
To draw with the pen, the main commands will be to:

1. Put the pen down (start writing)
2. Move (this is explained in the next slide - blue “motion” category)
3. Pick the pen up (stop writing)
4. Clear (delete everything written by the pen)



Pen Settings

You can also set the pen color (0-199) and pen size (width in pixels, 1-256).



Use these blocks
to set and change
the pen's ...

Hue

Saturation

Brightness

Transparency

More Pen Blocks



This block draws the
words/numbers in the
rectangular area.

Note: Your sprite is used to draw the letters. The letters will begin at your position and go to the right. Your sprite will end up to the right of the last letter.



This block draws
an image of your
sprite, at the
current location
with the pen.

Example: If your sprite is a yellow cat, it draws a yellow cat at your current position. That can't can't be changed later - only erased using the "clear" block.

Motion Blocks

Use Motion to define your location and movements.

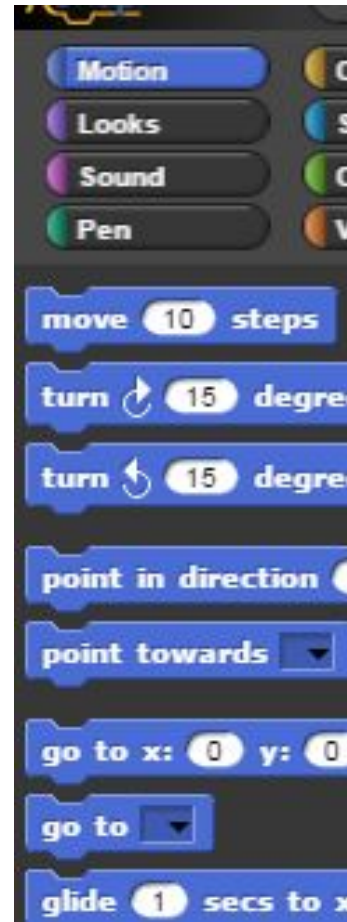
These motion blocks control the movement of sprites around the screen.

As a general rule:

Never leave assumptions in your code!

Always start with commands like “Go to (location)” and “Point in direction _____”.

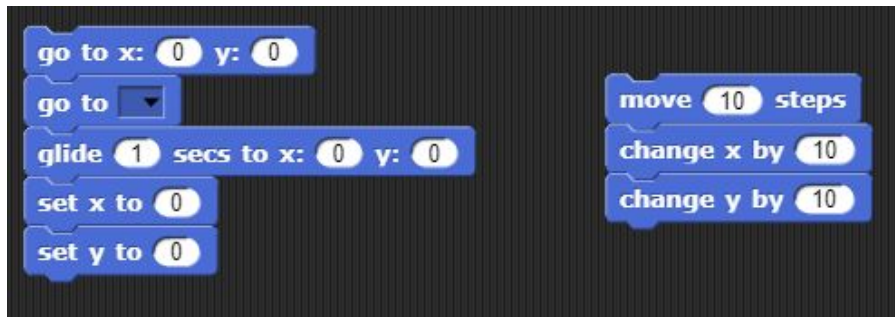
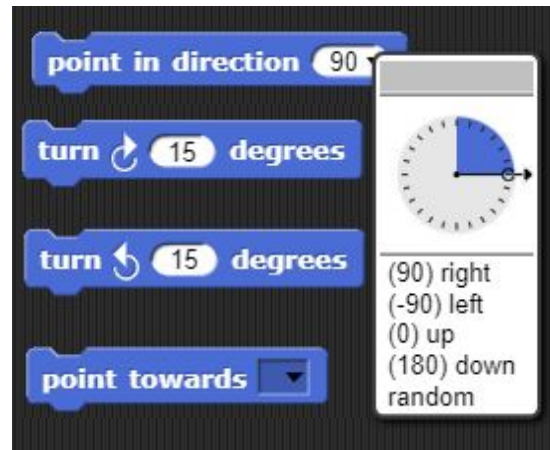
This greatly helps avoid glitches later on when you are using multiple collections of code together.



Motion Blocks

Direction - These are in degrees, with 0 pointing up. Positive degrees turn clockwise, and negative counter-clockwise.

You can also use the “Point towards ____” block to point towards another sprite.



Go / Move - “go to” or “set” blocks take you to specific locations. “Move” and “change” blocks move you a number of pixels from your current position.

Note: The glide block locks you into that motion, and no other code in the script runs until you finish gliding.

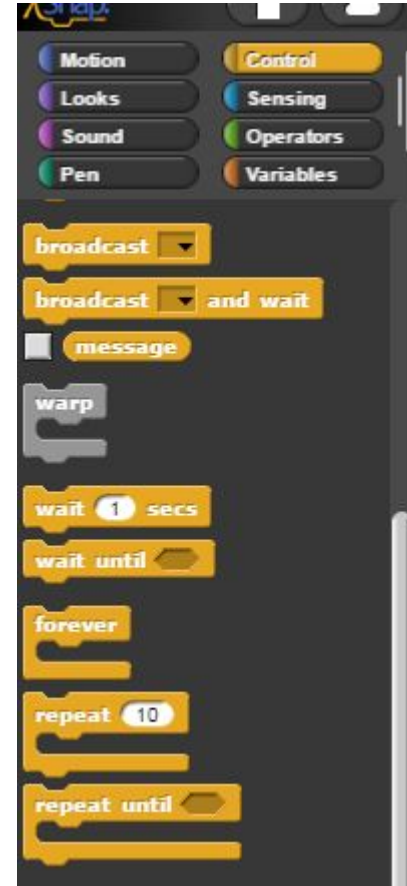
Repetition

We will use Control Blocks to repeat blocks of code.

These commands are the bread and butter of programming. They do not simply say “run ___”, but rather say when and how many times a block of code runs. It also controls when the code stops. Under what conditions? For how long?

These commands, along with clever data manipulation are what make up a lot of programming.

Otherwise, every single thing you wanted a computer to do, you would have to manually program each step. With repetition you can tell the computer ... “start doing this, and keep going until you are done”.



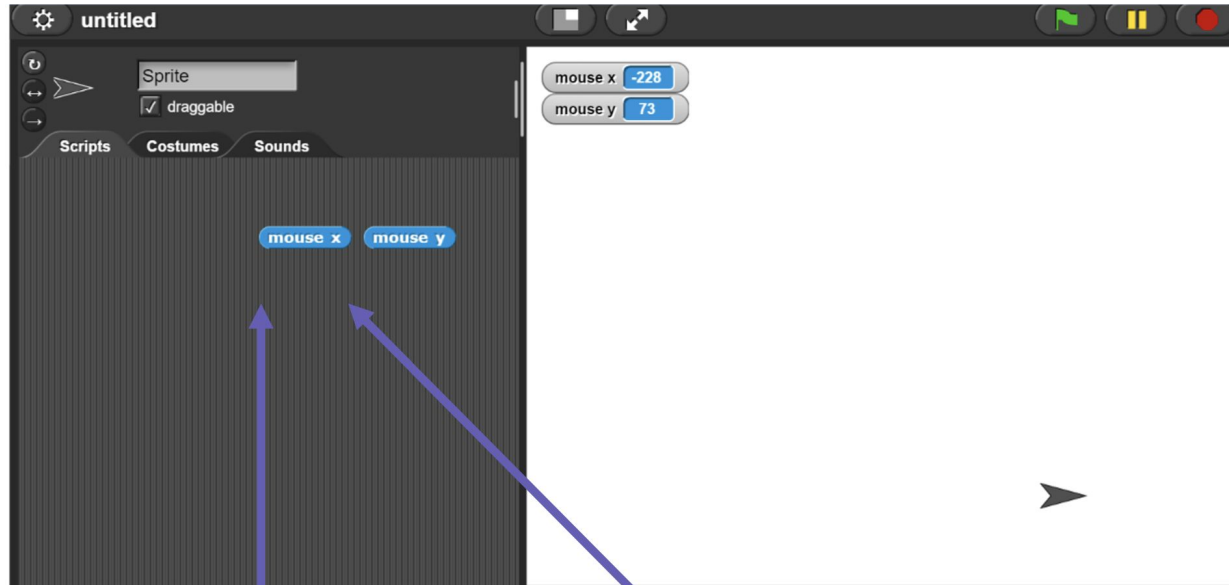


Variables



Intro to Snap!

Most things that we need to keep track of, we track with ***variables*** (named quantities)

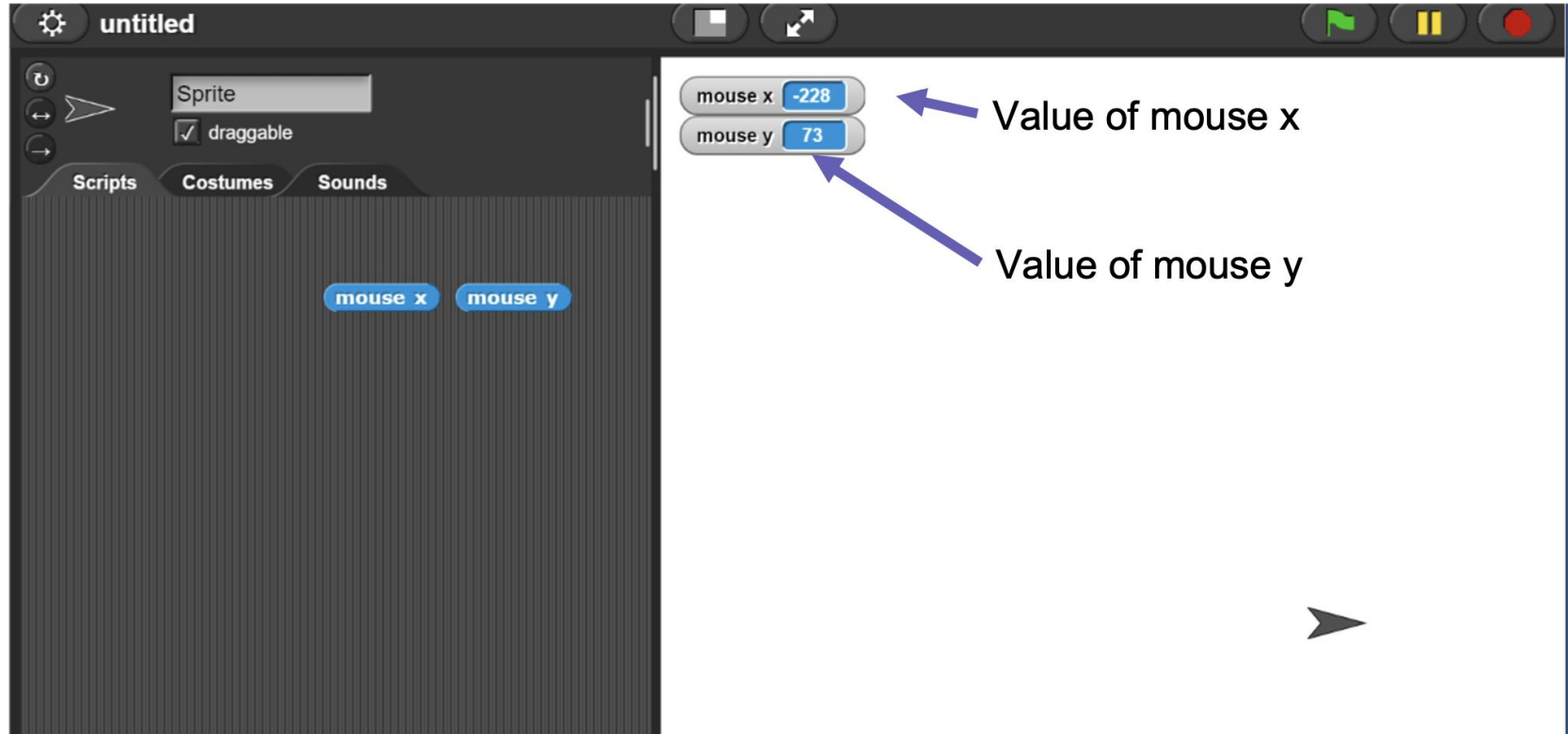


Variable: where on the
x axis is the mouse?

Variable: where on the
y axis is the mouse?

Intro to Snap!

Variables have a **name** and a **value**

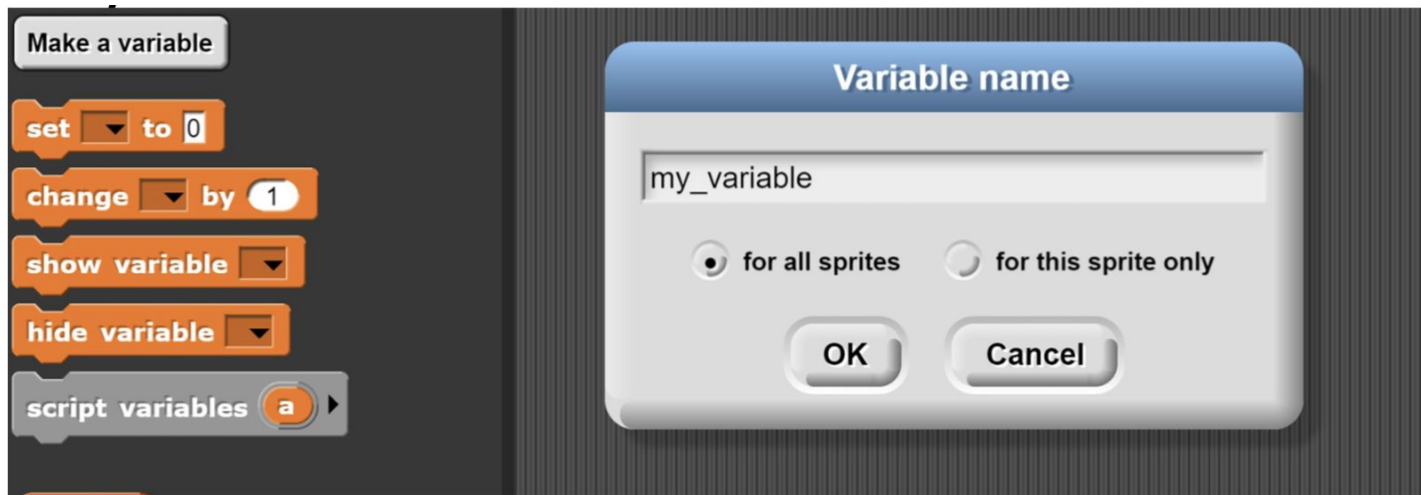


The screenshot shows the Snap! IDE interface. On the left, the 'Sprite' panel is visible with a 'Sprite' button and a 'draggable' checkbox. Below it are tabs for 'Scripts', 'Costumes', and 'Sounds'. In the 'Scripts' tab, there are two blue buttons labeled 'mouse x' and 'mouse y'. On the right, the main stage area shows two variable monitors. The first monitor is labeled 'mouse x' and has a value of '-228'. The second monitor is labeled 'mouse y' and has a value of '73'. Two blue arrows point from the text 'Value of mouse x' to the '-228' value, and from the text 'Value of mouse y' to the '73' value. The stage area also features a grey arrow cursor pointing towards the bottom right.



We can use variables in our code!

Some variables are built in (e.g., “answer” is the answer to a question in Snap). You can make your own variables:



Variables are (usually) in orange. Things that are black writing on white are constants – (The value remains as stated)





Q: What is the value of “my_variable” after the following code is run?



A. 6

B. 9

C. 42

D. 54

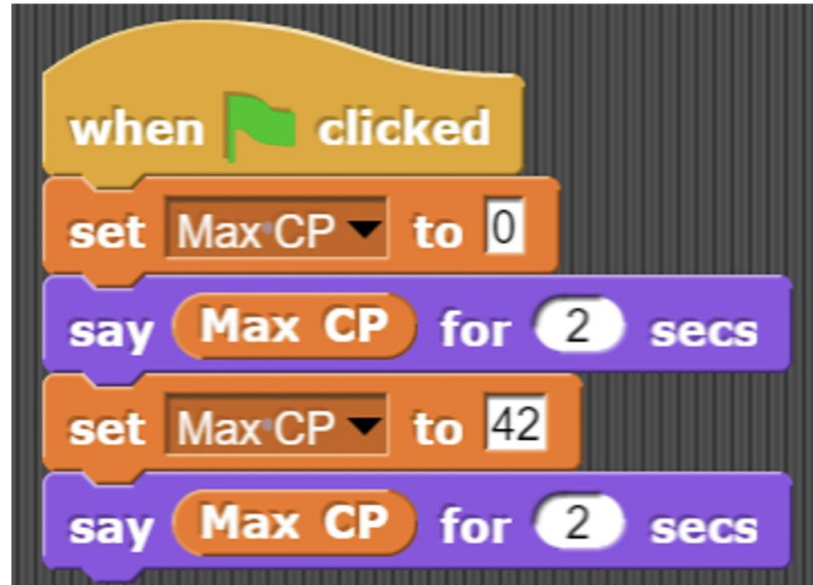
E. None of the above



Q: What is the value of “Max CP” after all lines of this program are executed?



- A. 0
- B. 2
- C. 42
- D. 042
- E. None of the above



Q: What is the value of “shoe size” after all lines of the program are executed?

- A. 2
- B. 39
- C. 40
- D. 3940



Class Activity!



Class Activity

- Use the online Snap! Interface: <http://snap.berkeley.edu/snap> to do the following:
 1. Change the costume from an arrow to something from the Media Library
 2. Have your Sprite go from the middle to the top left
 3. Then make it go to the bottom right
 4. Then to the bottom left
 5. Then to the bottom right
 6. On PrairieLearn, **submit 4 pictures** - one at each of the four corners!

Mutation



Mutation

Process of **changing the state** or data of an **object** after it has been created.

Repeat 10 times:

1. Preheat oven (400° C)
2. **Combine ingredients in bowl to form dough**
3. Put dough into bread pan
4. If ingredients contain yeast, allow to sit at room temperature for 1 hour
5. Put bread pans into preheated oven and bake for 30 minutes

Q: What is the value of “age” after all lines of this program are executed?



- A. 1
- B. 40
- C. 41
- D. 401

```

when clicked
  set age to 40
  say join Your age is now age for 2 secs
  set age to age + 1
  say join Happy Birthday! Your age is now age for 2 secs
  
```


Wrap up